

Agentic-Analyst Comprehensive Architecture & Logic Report

1. Executive Summary & System Architecture

The **Agentic-Analyst** is an enterprise-grade AI Data Assistant. It bridges the gap between natural language requests and complex SQL analytics, functioning as a multi-agent system.

- **Frontend:** A React-based interface that handles dynamic rendering of chats, tables, dashboards, and ERD diagrams.
- **Backend:** A Python FastAPI server.
- **AI Engine:** Powered locally by qwen2.5-coder:7b through Ollama, using extensive fallback heuristics for tunnel/cloud disruptions.
- **Database:** Connects to jghMasterDB via SQLAlchemy.

2. The Core Execution Pipeline (Agent Workflow)

When a user submits a natural language question, the system executes a strict, multi-step pipeline inside `sql_agent.py`.

1. Memory & Context Resolution (`memory_manager.py`)

- **Logic:** Keeps a sliding window of the last 10 conversational turns. It uses NLP to decide if the current question is a follow-up (inheriting previous tables, dates, and IDs) or a complete context reset.

2. Ambiguity Firewall (`ambiguity_checker.py`)

- **Logic:** Before hitting the AI, it checks if the question is dangerously vague (e.g., "Show August data"). If it detects missing entities or metrics, it halts execution and returns a multiple-choice list of clarifications to the frontend.

3. Intent Classification (`intent_router.py`)

- **Logic:** Uses RegEx to categorize the query into one of 5 execution modes: TUTOR_QA, ERD_GEN, DASHBOARD_GEN, BUSINESS_STORY, or default SQL_ANALYTICS.

4. Deterministic Query Planning (`query_planner.py`)

- **Logic:** Synthesizes a structured JSON "Execution Plan" from the natural language. It determines the primary entity, relationships, required columns, and GROUP BY logic before the AI ever sees the prompt.

5. LLM Execution & Parsing (`nlp_intent_parser.py` & `sql_generator.py`)

- **Logic:** Prompts the qwen2.5-coder model to output a strict JSON representation of the query, then converts that representation into a MySQL dialect string. If the LLM is offline, a deterministic fallback synthesizes basic SQL from the Query Plan.

6. Multi-Stage Validation (`sql_ast_validator.py` & `semantic_sql_validator.py`)

- **Logic:** Parses the generated SQL into an Abstract Syntax Tree (AST). It blocks destructive commands (DROP, DELETE, UPDATE) and strictly verifies that all tables and columns exist in the enterprise schema to prevent hallucinations.

7. Execution (`read_executor.py`)

- **Logic:** Safely executes the validated SELECT query against the database with a strict row limit to prevent memory overflow.

8. End-to-End Accuracy Validation (`e2e_accuracy_validator.py`)

- **Logic:** Inspects the final data payload against the original question. If the user asked for "highest earnings" and the SQL forgot the ORDER BY DESC, this validator blocks the result and forces a self-correction retry loop (up to 3 retries).

9. Natural Language Synthesis (`response_generator.py`)

- **Logic:** Translates the raw data array into a conversational, executive summary for the user to read.

3. Deep-Dive: Specific Feature Logic & Parameter Handling

A. Temporal & Date Boundaries

- **Logic Engine:** `QueryPlanner._resolve_temporal`
- **Features:** It translates relative human terms ("this month", "last month", "July 2026") into hard SQL constraints using Python's `datetime`. E.g., "this month" becomes `start_date = '2026-08-01', end_date = '2026-09-01'`.

B. Specific Identifier Lookups

- **Logic Engine:** `id_search.py (extract_id_from_prompt)`
- **Features:** Automatically scans prompts for 4-to-6 digit numeric IDs (e.g., 58997). It determines if the ID belongs to a distributor or retailer based on surrounding words, injecting `WHERE id = 58997` without requiring the AI to figure it out.

C. Automatic Entity & Role Resolution

- **Logic Engine:** `QueryPlanner`
- **Features:**
 - If a user mentions "retailer", it automatically injects a background filter `WHERE users.user_role = 2`.
 - "distributor" maps to `users.user_role = 4`.
 - "wholesaler" maps to `users.user_role = 5`.

D. Automated Metric Computations

- **Earnings / Revenue:** Automatically maps to the `wallet_transaction` table and triggers a `SUM(wallet_transaction.amount)` aggregation.
- **TDS / Tax:** Automatically maps to the `withdrawal_request` table.
- **Top / Highest / Lowest:** Injects `ORDER BY` and `LIMIT` clauses to truncate the dataset.

4. Specialized Collaborative Modes Logic

4.1 Dashboard Builder (`dashboard\_builder.py`)

- **Trigger:** "build dashboard", "analytics panel".
- **Logic Flow:** Instead of executing SQL, it generates a UI specification JSON (`dashboard_spec`).
- **Data Sources:** Currently uses **aggregated static snapshots** (e.g., hardcoded July 2026 transaction counts) to ensure instantaneous dashboard rendering.
- **Frontend Rendering:** The React app receives this JSON and iterates over the `metric_cards` and `chart_specs` arrays, feeding the data arrays into interactive Recharts pie/bar graph components.

4.2 Executive Storyteller (`storyteller.py`)

- **Trigger:** "executive summary", "business trends".
- **Logic Flow:** Avoids tables and charts entirely. It fetches high-level metrics, passes them to the LLM with a highly specific prompt instructing the model to act as a Chief Financial Officer, and outputs a purely narrative-driven markdown summary.

4.3 Database Tutor (`tutor_agent.py`)

- **Trigger:** "explain", "what is", "teach me".
- **Logic Flow:** Instead of querying user data, it queries the internal system metadata (`knowledge_graph.py`). It explains schemas, role structures, and technical terminology without exposing private data.

4.4 ERD Generator (`erd_generator.py`)

- **Trigger:** "ER diagram", "draw architecture".
- **Logic Flow:** Extracts the database schema and generates Mermaid.js syntax mapping primary and foreign key relationships. The frontend renders this syntax into an interactive visual graph.

5. Security & Safety Firewalls

1. **The Security Firewall:** Hardcoded checks block queries containing terms like "password", "encryption key", or "auth token", immediately denying access before the LLM even sees the request.
2. **Read-Only Executor:** `execute_read_query()` uses SQLAlchemy with restricted database credentials that physically cannot execute write/modify operations.
3. **Semantic Schema Validation:** Prevents the AI from querying hallucinated tables or non-existent columns by cross-referencing every generated query against an active schema map of `jghMasterDB`.